

Přijímací pohovor: Integrační tester

Informace pro uchazeče

Vítejte u praktické části pohovoru na pozici **integračního testera**. Vaším úkolem je otestovat mock službu **Interview Mock**, která spravuje pohovory a kandidáty. Služba nabízí **REST API** i **SOAP webovou službu**.

- **Časový limit:** 90 minut (praktická část) + 15 minut (teoretické otázky)
- **Nástroj:** Můžete si vybrat **SoapUI** nebo **Postman** (případně oba); pro SQL část stačí `sqlite3` v terminálu
- **Povinné:** Musíte otestovat **REST i SOAP** rozhraní bez ohledu na zvolený nástroj

Co dělá integrační tester (pro jistotu)

Ověřuje, že **dva nebo více systémů spolu mluví správně** — typicky volá API, kontroluje odpovědi, hledá rozdíly mezi specifikací a skutečností. V pohovoru budete:

1. Posílat HTTP požadavky na REST a SOAP endpointy.
2. Kontrolovat status kódy, hlavičky, tělo odpovědi.
3. Zkoušet i **negativní scénáře** (špatný klíč, chybějící data, neplatné hodnoty) — od testera se očekává, že systém umí selhat *kontrolovaně*.
4. Psát SQL dotazy nad ukázkovou databází.

Slovníček pojmů (pokud na něčem zaváháte)

Pojem	Co to je
Endpoint	Konkrétní adresa služby, např. <code>GET /rest/interviews</code> . Skládá se z HTTP metody (GET/POST/...) a cesty.
HTTP metoda	GET = čti, POST = vytvoř, PATCH = částečná změna, PUT = nahraď celé, DELETE = smaž.
Status kód	Třímístné číslo v odpovědi: 2xx = úspěch, 4xx = chyba klienta, 5xx = chyba serveru.
Header (hlavička)	Metadata HTTP zprávy, např. <code>Content-Type: application/json</code> nebo <code>X-API-Key:</code>
Body (tělo)	Skutečná data v requestu/response — JSON u REST, XML u SOAP.
API klíč	Tajný řetězec, kterým se klient autentizuje. Posílá se v hlavičce <code>X-API-Key</code> .
REST	Architektonický styl webových API. Používá HTTP metody nad zdroji (např. <code>/interviews/{id}</code>), data typicky v JSON.
SOAP	Protokol pro webové služby založený na XML. Volá se vždy POST na jeden endpoint a operace je určená v těle XML zprávy + hlavičkou <code>SOAPAction</code> .
WSDL	“Návod” k SOAP službě — XML soubor popisující, jaké operace existují a jaké mají parametry.
Idempotence	Když opakované volání má stejný efekt jako jedno volání (GET, PUT, DELETE). POST idempotentní není — každé volání vytvoří nový záznam.
Mock	Falešná verze služby pro testování. Tady jste přesně v takové.

Spuštění služby

Služba běží na adrese, kterou vám sdělí zkoušející (výchozí: `http://localhost:4010`).

Užitečné odkazy: - **REST dokumentace (OpenAPI)**: otevřete v prohlížeči `http://localhost:4010/openapi.yaml`
- **SOAP WSDL**: otevřete `http://localhost:4010/soap?wsdl` - **Healthcheck**: `http://localhost:4010/health`
— pokud vrátí JSON `{"status":"ok", ...}`, je vše OK

Než začnete (5 minut na rozkoukání)

1) *Ověřte, že služba odpovídá:*

```
curl http://localhost:4010/health
```

2) *Otevřete OpenAPI specifikaci (REST kontrakt) a WSDL (SOAP kontrakt)*

a 1–2 minuty si je proklikněte/projděte.

Autentizace (klíče, role)

Služba vyžaduje **API klíč** v hlavičce `X-API-Key` pro většinu operací. Bez klíče dostanete 401 Unauthorized, se špatnou rolí 403 Forbidden.

Klíč	Role	Co smí
interview-key-2026	admin	čtení i zápis (POST/PATCH)
readonly-key-2026	reader	pouze čtení (GET)

Endpointy bez nutnosti autentizace: `/`, `/health`, `/openapi.yaml`, `/rest`, `GET /soap?wsdl`

Příklad volání s klíčem:

```
curl -H "X-API-Key: interview-key-2026" http://localhost:4010/rest/interviews
```

V Postmanu klíč nastavte jako **environment variable** `apiKey` a v záložce Headers každého requestu napište `X-API-Key: {{apiKey}}`. V SoapUI použijte property na úrovni Test Suite nebo Test Case.

Část 1: Praktické úlohy (90 minut)

Tip k assertion: U každého kroku se očekává, že v nástroji nastavíte **kontrolu** (assertion, test script), ne jen vizuální oko. Postman: záložka *Tests*, kde napíšete `pm.test("status 200", () => pm.response.to.have.status(200))`; SoapUI: na požadavku pravým tlačítkem → *Add Assertion* → např. *Valid HTTP Status Codes*, *Simple Contains*, *XPath Match*.

Úloha A: Zabezpečení API (15 minut)

Co se ověřuje: že server správně rozlišuje mezi „nejsi přihlášený“ (401) a „přihlášený jsi, ale nemáš oprávnění“ (403), a že veřejné endpointy klíč nepožadují.

1. **Volání bez API klíče** — zavolejte `GET /rest/interviews` **bez** hlavičky `X-API-Key`. Očekáváno: 401 Unauthorized, tělo obsahuje chybu typu UNAUTHORIZED.

```
curl -i http://localhost:4010/rest/interviews
```

2. **Neplatný API klíč** — stejný endpoint s klíčem `invalid`. Očekáváno: opět 401.

```
curl -i -H "X-API-Key: invalid" http://localhost:4010/rest/interviews
```

3. **Platný admin klíč** — `GET /rest/interviews` s `interview-key-2026`. Očekáváno: 200 a JSON s polem `interviews`.

4. **Read-only klíč na čtení** — stejné volání s `readonly-key-2026`. Očekáváno: 200, čtení funguje.
5. **Read-only klíč na zápis** — zkuste `POST /rest/interviews` s `readonly-key-2026`. Očekáváno: **403 Forbidden** s chybou `FORBIDDEN`. (Klíč je platný, ale role nesmí zapisovat.)

```
curl -i -X POST -H "X-API-Key: readonly-key-2026" \
  -H "content-type: application/json" \
  -d '{"candidateId":"CAND-001","position":"x"}' \
  http://localhost:4010/rest/interviews
```
6. **Veřejné endpointy** — ověřte, že `/health` a `GET /soap?wsdl` fungují **bez** API klíče. (Pokud zde server vyžaduje klíč, je to chyba implementace.)
7. **SOAP autentizace** — ověřte, že i SOAP operace `ListInterviews` (`POST /soap`) vyžaduje klíč. Volání bez klíče má vrátit 401.

Úloha B: REST API (20 minut)

Poznámka: Pro všechny následující requesty použijte admin klíč `X-API-Key: interview-key-2026`.

Co se ověřuje: umíte projít typický CRUD (číst, vytvořit, upravit) + správně vyložit status kódy a strukturu odpovědi.

1. **Healthcheck** — `GET /health`. Očekáváno: 200, tělo `{"status":"ok","service":"interview-mock",...}`.
2. **Seznam pohovorů** — `GET /rest/interviews`. Ověřte, že:
 - status je 200
 - tělo má pole `interviews` (array)
 - každý záznam má alespoň `id`, `status`, `candidate`

```
curl -H "X-API-Key: interview-key-2026" http://localhost:4010/rest/interviews
```

3. **Filtrování podle statusu** — `GET /rest/interviews?status=SCHEDULED`. Všechny vrácené záznamy mají mít status `SCHEDULED`. Pro každý zapnutý záznam tedy `assertion: interviews[*].status == "SCHEDULED"`.
4. **Detail pohovoru** — `GET /rest/interviews/INT-001`. Tělo má obsahovat objekt `interview` a v něm `candidate` se `firstName`, `lastName`, `email`, `skills`.
5. **Vytvoření nového pohovoru** — pošlete `POST /rest/interviews` s JSON tělem:

```
{ "candidateId": "CAND-002",
  "position": "Integration Tester",
  "scheduledAt": "2026-06-01T10:00:00.000Z" }
```

Očekáváno: 201 Created. **Z odpovědi si poznamenejte `interview.id`** — budete ho potřebovat v dalších krocích.

6. **Změna statusu** — `PATCH /rest/interviews/{ID_Z_KROKU_5}/status` s tělem `{"status":"IN_PROGRESS"}`. Očekáváno: 200, v odpovědi `interview.status == "IN_PROGRESS"`.
7. **Hodnocení pohovoru** — `POST /rest/interviews/{ID_Z_KROKU_5}/evaluate` s tělem `{"score": 68}`. Očekáváno:
 - 200
 - `interview.status == "COMPLETED"`
 - `interview.recommendation == "REVIEW"` (mock dává `HIRE ≥ 80`, `REVIEW` pro 55–74, `NO_HIRE` níže)
8. **Negativní testování (min. 2 scénáře)** — vyzkoušejte, že server správně **odmítne**:
 - **Neexistující pohovor:** `GET /rest/interviews/INT-999` → 404, tělo obsahuje `not found`
 - **Neexistující kandidát při vytváření:** `POST /rest/interviews` s `candidateId: "CAND-XXX"` → 400

- **Nevalidní skóre:** POST `.../evaluate` se skóre: 150 → 400
- **Neplatný status:** PATCH `.../status` s status: "B0GUS" → 400

Stačí provést dva z těchto čtyř scénářů (a získat oba body).

Úloha C: Řetězení parametrů a práce s proměnnými (15 minut)

Co se ověřuje: umíte si vytáhnout hodnotu z odpovědi a použít ji v dalším requestu, a chápete **rozdíly mezi úrovněmi proměnných**.

Rozlišujte, **kam** proměnnou uložíte: - **Globální** — viditelné napříč všemi kolekcemi/projekty. Vhodné jen pro opravdu obecné konstanty. - **Environment** — specifické pro prostředí (test vs. produkce). Sem patří `baseUrl`, `apiKey`. - **Collection / lokální** — platné jen v rámci jedné kolekce nebo jednoho requestu. Sem patří hodnoty získané za běhu (`candidateId`, `interviewId`).

Zkoušející se vás na konci zeptá, **proč jste kterou proměnnou uložili na danou úroveň**.

1. **Nastavte prostředí** — vytvořte environment „Test“ a uložte do něj:

- `baseUrl = http://localhost:4010`
- `apiKey = interview-key-2026`

Použijte je ve všech requestech této úlohy jako `{{baseUrl}}` a `{{apiKey}}`.

2. **Najděte kandidáta podle emailu** — GET `{{baseUrl}}/rest/candidates?email=petr.svoboda@example.test`. V odpovědi je `candidate.id`. Uložte ho do **collection proměnné** `candidateId`:

- **Postman** (záložka *Tests*):

```
const data = pm.response.json();
pm.collectionVariables.set("candidateId", data.candidate.id);
```

- **SoapUI:** přidejte *Property Transfer* step, source = odpověď tohoto requestu (`//candidate/id`), target = property na úrovni TestSuite.

3. **Vytvořte pohovor pro nalezeného kandidáta** — POST `{{baseUrl}}/rest/interviews` s tělem:

```
{ "candidateId": "{{candidateId}}",
  "position": "Senior Integration Tester",
  "scheduledAt": "2026-07-01T09:00:00.000Z" }
```

Žádné hardcoded ID! Uložte si `interview.id` do proměnné `interviewId`.

4. **Ověřte vytvoření** — GET `{{baseUrl}}/rest/interviews/{{interviewId}}`. Zkontrolujte, že detail patří správnému kandidátovi (`interview.candidate.id == {{candidateId}}`).
5. **Proveďte celý lifecycle** s `{{interviewId}}`:
 - PATCH `.../status` na `IN_PROGRESS`
 - POST `.../evaluate` se skóre 91
 - Ověřte, že `interview.recommendation == "HIRE"`
6. **Ověřte výsledek přes filtr** — GET `{{baseUrl}}/rest/interviews?candidateId={{candidateId}}&status=COMPLETED`. Nový pohovor musí být v seznamu.
7. **Najděte kandidáta podle skillu** — GET `{{baseUrl}}/rest/candidates?skill=SQL`. Zkontrolujte počet vrácených kandidátů.

Úloha D: SOAP API (15 minut)

Co se ověřuje: umíte si sestavit SOAP envelope, zavolat operaci, přečíst odpověď i SOAP Fault.

SOAP je **vždy POST na jednu URL** (zde `/soap`). Operace se určuje XML elementem v těle a hlavičkou `SOAPAction`. Hlavička `Content-Type` musí být `text/xml`; `charset=utf-8`.

Šablona envelope (vyplníte si jméno operace a parametry):

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:NAZEV_OPERACE_Request>
      <!-- parametry -->
    </tns:NAZEV_OPERACE_Request>
  </soap:Body>
</soap:Envelope>
```

Vždy posílejte tyto hlavičky: - Content-Type: text/xml; charset=utf-8 - SOAPAction: urn:interview-mock#NAZEV_OPERACE - X-API-Key: interview-key-2026

1. **Získejte WSDL** — GET /soap?wsdl. Najděte 4 operace: ListInterviews, GetInterview, CreateInterview, UpdateInterviewStatus.
2. **Výpis pohovorů** — operace ListInterviews (tělo <tns:ListInterviewsRequest/>). Odpověď musí být XML, najděte v něm aspoň jeden INT-... záznam.
3. **Detail pohovoru** — operace GetInterview pro INT-002:

```
<tns:GetInterviewRequest>
  <tns:id>INT-002</tns:id>
</tns:GetInterviewRequest>
```

Ověřte, že odpověď obsahuje <score>82</score> a <recommendation>HIRE</recommendation>.

4. **Vytvoření pohovoru přes SOAP** — operace CreateInterview:

```
<tns:CreateInterviewRequest>
  <tns:candidateId>CAND-001</tns:candidateId>
  <tns:position>SOAP Tester</tns:position>
  <tns:scheduledAt>2026-06-15T14:00:00.000Z</tns:scheduledAt>
</tns:CreateInterviewRequest>
```

5. **Změna statusu přes SOAP** — operace UpdateInterviewStatus pro pohovor, který jste vytvořili v kroku 4, na status CANCELLED.

Tip — co je SOAP Fault: pokud server vrátí chybu, dostanete <soap:Fault> s elementem <fault-string> a <detail> s chybovým kódem (INTERVIEW_NOT_FOUND, UNKNOWN_CANDIDATE, INVALID_STATUS). Status kód HTTP bývá 400 nebo 404. **Není to selhání nástroje — je to validní SOAP odpověď.**

Úloha E: Porovnání a reporting (10 minut)

1. **Cross-protocol ověření** — ověřte, že pohovor vytvořený přes SOAP (úloha D, krok 4) je viditelný i přes REST: GET /rest/interviews. Najděte ho v seznamu. Stejně tak pohovor vytvořený přes REST (úloha B, krok 5) musí být viditelný přes SOAP ListInterviews. To dokazuje, že obě API sdílejí stejná data.
2. **Shrnutí** — řekněte zkoušejícímu na 2-3 minuty:
 - Kolik testů jste provedli a kolik prošlo?
 - Našli jste **něco podezřelého**? (např. neenforcovaná auth, chybějící endpoint, divná chybová zpráva)
 - Jaké testy byste přidali, kdybyste měli další hodinu?
 - Co byste navrhli pro zlepšení **zabezpečení** (rate limit, HTTPS, JWT místo statického klíče, audit log...)?

Úloha F: SQL nad lokální databází (15 minut)

Co se ověřuje: umíte základní SQL (JOIN, GROUP BY, HAVING, LEFT JOIN, transakce). Databáze **není napojená na běžící mock** — slouží k procvičení dotazů.

Než začnete:

```
cd sql
sqlite3 -header -column interview.db
# ocitnete se v promptu sqlite>; ukončete: .quit nebo Ctrl-D
```

Užitečné . příkazy uvnitř sqlite3: - .tables — vypíše všechny tabulky - .schema candidates — schéma konkrétní tabulky - .headers on + .mode column — pěkné formátování

Tabulky a vztahy (viz sql/README.md):

```
companies <- positions <- interviews >- candidates <- candidate_skills >- skills
                                     |
                                     <- interview_status_history
                                     <- evaluations >- recruiters
```

Napište a spusťte SQL pro každý úkol. Výsledek ukažte nebo zkopírujte zkoušejcímu.

1. **Kandidáti se skilem SQL** — vypište id, celé jméno (sloučení first_name a last_name) a level (z candidate_skills.level). Seřadte tak, aby experti byli první. *Hint: potřebujete JOIN candidates s candidate_skills a skills, filtr skills.name = 'SQL'.*
2. **Počty pohovorů podle statusu** — SELECT status, COUNT(*) ... GROUP BY status. Vrátí 5 řádků (SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED, FAILED).
3. **Aktivní kandidáti (≥ 2 pohovory)** — GROUP BY candidate_id + HAVING COUNT(*) >= 2. Pozor: WHERE COUNT(*) >= 2 **nefunguje** — WHERE se aplikuje *před* agregací, HAVING *po* ní.
4. **Kandidáti bez pohovoru** — LEFT JOIN candidates → interviews, pak WHERE interviews.id IS NULL. INNER JOIN by tento výsledek vrátit nemohl.
5. **Průměrné skóre podle pozice** — pro pohovory status = 'COMPLETED' vypište positions.title, COUNT(*) a AVG(score). Sestupně podle průměru.
6. **Top 3 technická hodnocení** — z tabulky evaluations vypište 3 nejvyšší score v kategorii technical spolu s celým jménem kandidáta a recruitera. To je 4-tabulkový JOIN: evaluations → interviews → candidates, + evaluations.evaluator_id → recruiters.

Bonus (volitelné, +5 b): otevřete transakci, vložte nový pohovor (INSERT INTO interviews ...) **a zároveň** zapište přechod do interview_status_history. Pak ROLLBACK — ověřte, že obě změny zmizely (atomicita).

BEGIN;

```
INSERT INTO interviews (id, candidate_id, position_id, recruiter_id, scheduled_at, status)
VALUES ('INT-TEST', 'CAND-001', 1, 1, '2026-12-01T10:00:00.000Z', 'SCHEDULED');
```

```
INSERT INTO interview_status_history (interview_id, old_status, new_status, changed_at, changed_by)
VALUES ('INT-TEST', NULL, 'SCHEDULED', '2026-12-01T10:00:00.000Z', 1);
```

```
SELECT id FROM interviews WHERE id = 'INT-TEST'; -- uvnitř TX: jeden řádek
```

ROLLBACK;

```
SELECT id FROM interviews WHERE id = 'INT-TEST'; -- po rollback: prázdko
```

Část 2: Teoretické otázky (15 minut)

Zkoušejcí vybere 6-8 otázek z následujícího seznamu (ideálně 2 REST + 1-2 SOAP + 1-2 SQL + 1-2 obecné/security).

Tip pro odpověď: stručně a věcně. Lepší je říct „nevím přesně, ale myslím, že ...“ než hádat se sebejistou nepřesností. Pokud tušíte odpověď z praxe, dejte konkrétní příklad.

REST API

1. Co znamená zkratka REST a jaké jsou základní principy RESTful API?
2. Jaký je rozdíl mezi HTTP metodami PUT a PATCH?

3. Co je to idempotence a které HTTP metody by měly být idempotentní?
4. Vysvětlete skupiny HTTP status kódů (1xx, 2xx, 3xx, 4xx, 5xx) a uveďte příklady.
5. Co je OpenAPI (Swagger) specifikace a k čemu slouží?
6. Jak byste testovali autentizaci a autorizaci REST API (API klíče, OAuth, JWT)?
7. Co je to Content Negotiation v kontextu HTTP?

SOAP

8. Co znamená zkratka SOAP a jaké jsou hlavní části SOAP zprávy (Envelope, Header, Body)?
9. Co je WSDL a jaké informace obsahuje?
10. Jaký je rozdíl mezi SOAP styly document/literal a rpc/encoded?
11. Jak se v SOAP komunikují chyby (SOAP Fault)? Jakou strukturu má Fault element?
12. Jaké jsou výhody a nevýhody SOAP oproti REST?

SQL

Volitelná praktická verifikace: v `sql/interview.db` je připravená SQLite databáze (40 kandidátů, 50 pohovorů, plně normalizované schéma). Uchazeč si může otázky 22–25 ověřit reálně: `sqlite3 -header -column sql/interview.db, nebo sqlite3 sql/interview.db < sql/examples.sql` pro ukázkové dotazy.

22. Jaký je rozdíl mezi INNER JOIN, LEFT JOIN a FULL OUTER JOIN? Kdy který použijete?
23. K čemu slouží klauzule GROUP BY a co je agregační funkce? Uveďte 3 příklady (např. COUNT, SUM, AVG).
24. Jaký je rozdíl mezi klauzulemi WHERE a HAVING?
25. Jaký je rozdíl mezi UNION a UNION ALL? Kdy je vhodné použít každou z nich?
26. Vysvětlete pojmy primary key, foreign key a unique constraint. Co se stane při pokusu o vložení duplicitní hodnoty?
27. Jaký je rozdíl mezi DELETE, TRUNCATE a DROP? Která z těchto operací je nevratná a která transakční?
28. Co je SQL injection a jak se mu jako tester/vývojář dá bránit (parametrizované dotazy, ORM, vstupní validace)?
29. Co jsou transakce a co znamená zkratka ACID? K čemu slouží COMMIT a ROLLBACK?

Zabezpečení a obecné

13. Co je to integrační testování a čím se liší od unit testování a E2E testování?
14. Jaký je rozdíl mezi autentizací a autorizací? Uveďte příklady z praxe.
15. Co je to contract testing (např. Pact) a kdy je vhodné ho použít?
16. Jak byste testovali API s ohledem na výkon (zátěžové testy, response time)?
17. Jaký je rozdíl mezi pozitivním a negativním testováním? Uveďte příklady pro API.
18. Co jsou to mocky a stuby a kdy je vhodné je použít při integračním testování?
19. Jaké znáte způsoby zabezpečení API? Porovnejte API klíče, OAuth 2.0 a JWT.
20. Co je to CORS a proč je důležitý pro webové API?
21. Jaký je rozdíl mezi globálními, environment a collection proměnnými v Postmanu (nebo odpovídajícími úrovněmi v SoapUI)? Kdy použijete kterou úroveň?

Hodnocení

Praktická část (max. 85 bodů)

Kritérium	Body
Zabezpečení: Ověření autentizace a autorizace (úloha A)	10
REST: Úspěšné provedení kroků 1-7 (úloha B)	15
REST: Negativní testování, min. 2 scénáře (úloha B)	5
Řetězení parametrů: Dynamické použití dat z odpovědi (úloha C)	15

Kritérium	Body
SOAP: Úspěšné provedení kroků 1-5 (úloha D)	15
Cross-protocol ověření (úloha E)	5
SQL: Dotazy 1-6 nad lokální databází (úloha F)	15
SQL: Bonus — transakce s rollbackem (úloha F)	+5
Kvalita práce (struktura requestů, assertions, organizace)	5

Teoretická část (max. 40 bodů)

Kritérium	Body
REST otázky (5 bodů za otázku)	max. 10
SOAP otázky (5 bodů za otázku)	max. 10
Zabezpečení a obecné otázky (5 bodů za otázku)	max. 10
SQL otázky (5 bodů za otázku)	max. 10

Celkové hodnocení

Maximum 125 bodů (85 praktická + 40 teoretická), bonus za SQL transakci se neevduje do stropu.

Výsledek	Procenta	Body
Vynikající	80–100 %	100–125
Dobry	60–79 %	75–99
Dostatečný	45–59 %	56–74
Nedostatečný	< 45 %	< 56

Časté chyby (čemu se vyhnout)

- **Hardcoded ID v requestech** v úloze C — když přepíšete `{{candidateId}}` číslem, ztratíte body za řetězení.
- **Žádné assertions** — testy bez kontrol jsou jen volání. V Postmanu *Tests*, v SoapUI *Assertions*.
- **Chybějící hlavička Content-Type** u POST/PATCH s JSON tělem → server může vrátit 415 *Unsupported Media Type*.
- **U SOAP zapomenutá SOAPAction** → server nepozná operaci a vrátí *Fault*.
- **Záměna 401 vs. 403** v úloze A — 401 = neznámý/chybějící klíč, 403 = známý klíč, ale špatná role.
- **WHERE COUNT(*) >= 2** v SQL — patří tam *HAVING*.

Nápovědy pro SOAP requesty

Struktura SOAP Envelope

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:OPERATION_NAME>
      <!-- parametry -->
    </tns:OPERATION_NAME>
  </soap:Body>
</soap:Envelope>
```


Příklad: GetInterview

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:GetInterviewRequest>
      <tns:id>INT-001</tns:id>
    </tns:GetInterviewRequest>
  </soap:Body>
</soap:Envelope>
```

Poznámka: SOAP requesty posílejte metodou POST na endpoint /soap s Content-Type: text/xml; charset=utf-8 a hlavičkou X-API-Key.